

Local Search

Lecture 7

CS 4880

Kirk Duran

Today

- ▶ Why path-based tree search can be the wrong abstraction for large optimization problems.
- ▶ How local search represents a problem using a complete state, an objective function, and a neighborhood.
- ▶ How hill climbing improves a single current state and why it can become trapped.
- ▶ How random restarts and simulated annealing introduce exploration.
- ▶ How local beam search and genetic algorithms search with populations of candidate states.
- ▶ Why local search remains useful in classical AI, combinatorial optimization, robotics, and games.

Key idea

Local search gives up explicit path construction in order to search enormous state spaces with very little memory.

Part 1: When Paths Stop Mattering

From Path Finding to State Optimization

Systematic search asks which path connects an initial state to a goal.

- ▶ DFS explores deeply before backtracking.
- ▶ BFS explores level by level.
- ▶ A prioritizes states using path cost and heuristic guidance.
- ▶ Each method maintains enough information to reconstruct a path.

A different question

What if we care primarily about the *quality of the final state*, not the sequence of actions used to reach it?

Image placeholder

Contrast a search tree/path with a single candidate configuration, such as a partially scrambled cube or optimization state.

Key idea

Optimization problems often need a good final configuration rather than an explicitly stored route through the state space.

When Tree Search Fails

Tree search can become inefficient or infeasible when:

- ▶ the state space is vast or effectively infinite;
- ▶ only the final state matters;
- ▶ the problem is naturally phrased as optimization rather than navigation;
- ▶ storing visited states, predecessors, and complete paths consumes too much memory.

Rubik's Cube illustration

If each position has roughly 18 legal moves, a naive tree has about 18 nodes at depth 1, 324 at depth 2, and 5,832 at depth 3.

Image placeholder

Exponentially expanding search tree beside a Rubik's Cube; label branching factor approximately 18.

Key idea

Exponential growth makes explicit search trees unattractive when the path itself provides little value.

Local Search

Definition

Local search operates from one or more current states and improves them using neighboring states, without explicitly storing complete paths through the search space.

- ▶ Evaluate the current state.
- ▶ Generate or sample neighboring states.
- ▶ Move according to an improvement or exploration rule.
- ▶ Repeat until a solution or stopping condition is reached.

Why use it?

Local search uses very little memory and can find good solutions even when systematic search is impractical.

Image placeholder

Single current-state node surrounded by neighboring states; arrows emphasize local movement rather than a stored tree.

Key idea

Local search is not systematic: it may never visit large portions of the state space, including regions containing solutions.

A New Problem: Eight Queens

Place eight queens on an 8×8 chessboard so that no two queens attack one another.

Complete-state formulation

Keep exactly one queen in each column. A state specifies the row occupied by the queen in every column.

- ▶ Every candidate already contains all eight queens.
- ▶ Most candidates are invalid because queens attack one another.
- ▶ Search improves the arrangement until all conflicts disappear.

Image placeholder

Two 8-queens boards: one conflicted configuration and one valid non-attacking configuration.

Key idea

Local search often starts with a complete but imperfect solution and iteratively improves it.

Part 2: Objective Functions and State-Space Landscapes



Objective Function

Definition

An **objective function** assigns a real-valued quality score to a complete candidate state:

$$f(s) \in \mathbb{R}.$$

- ▶ A local-search problem may seek to **minimize** or **maximize** this value.
- ▶ The score provides graded feedback even when the current state is not yet a solution.
- ▶ The neighborhood function determines which states can be reached by one local move.

Optimization model

State space + objective function + neighborhood function.

Image placeholder

Objective-function diagram for 8 queens showing attacking queen pairs and a conflict score.

Eight Queens as Optimization

For the complete-state formulation:

- ▶ **State:** eight queens, one queen per column.
- ▶ **Objective:** number of attacking pairs of queens.
- ▶ **Goal:** minimize the objective to 0.
- ▶ **Neighbor:** move one queen to a different row in its column.

Each of 8 queens can move to 7 other rows, giving

$$8 \cdot 7 = 56$$

immediate neighboring states.

Image placeholder

One 8-queens board with arrows showing a single queen moving within its column; annotate 56 possible neighbors.

Key idea

The neighborhood defines the algorithm's local view of the optimization landscape.

State-Space Landscape

A **state-space landscape** visualizes candidate states according to their objective values.

- ▶ Each point represents a complete state.
- ▶ Height represents objective value.
- ▶ Neighboring points correspond to locally reachable states.
- ▶ Local search moves across this landscape by choosing neighboring states.

Direction depends on the problem

For maximization, “uphill” is better. For minimization, the same picture can be interpreted by reversing the objective or descending toward lower cost.

Image placeholder

3D objective landscape labeled current state, shoulder, local maximum, flat local maximum, and global maximum.

Key idea

Local-search performance depends strongly on the topography induced by the objective and neighborhood functions.

Part 3: Hill Climbing — Repeated Local Improvement



Hill Climbing

Definition

Hill climbing repeatedly moves from the current state to its best improving neighbor.

- ▶ Keep only a single current state.
- ▶ Evaluate the immediate neighborhood.
- ▶ Move in the direction of greatest improvement.
- ▶ Stop when no neighboring state improves the objective.

Memory

Hill climbing does not need to store the path, previously visited states, or a large frontier.

Image placeholder

Objective surface with a trajectory climbing from a low point to a nearby peak.

Key idea

Hill climbing is a greedy local optimizer: it uses immediate improvement without planning several moves ahead.

Hill-Climbing Pseudocode

Algorithm

```
current ← initial_state
while true:
    neighbors ← NEIGHBORS(current)
    next ← best state in neighbors
    if next does not improve current:
        return current
    current ← next
```

- ▶ For maximization, “best” means highest objective value.
- ▶ For minimization, “best” means lowest objective value.
- ▶ The returned state may be a global optimum, a local optimum, or a plateau state.

Key idea

The algorithm’s simplicity is also its weakness: a one-step local decision cannot see a better region separated by worse intermediate states.

Hill Climbing on Eight Queens

Use the number of attacking pairs as the cost $h(s)$.

- ▶ Start with eight queens, one per column.
- ▶ Generate the 56 states obtained by moving one queen within its column.
- ▶ Choose the neighbor with the fewest attacking pairs.
- ▶ Repeat until $h(s) = 0$ or no improvement exists.

Goal state

$$h(s) = 0$$

means that no pair of queens attacks one another.

Image placeholder

8-queens board over a contour landscape; show one queen move reducing the number of attacking pairs.

Why Hill Climbing Gets Stuck

The state-space landscape can contain structures that defeat purely greedy improvement.

- ▶ **Local maximum:** better than every neighbor but worse than a distant state.
- ▶ **Ridge:** improvement requires coordinated movement not captured by one simple local step.
- ▶ **Plateau:** a flat region in which many neighboring states have equal value.
- ▶ **Shoulder:** a flat region from which an improving move eventually becomes available.

Image placeholder

Landscape labeled local maximum, global optimum, ridge, plateau, shoulder, and flat local maximum.

Key idea

A locally optimal state need not be globally optimal.

Sideways Moves

A **sideways move** transitions to a neighboring state with the same objective value.

- ▶ It neither improves nor worsens the current solution.
- ▶ It can move across a plateau until an improving exit is found.
- ▶ It can also wander indefinitely through a flat region.

Practical safeguard

Limit the number of consecutive sideways moves; the source example suggests a cap such as 100.

Image placeholder

Plateau landscape with a sideways trajectory, an eventual uphill exit, and a loop-risk annotation.

Key idea

Allowing neutral moves expands local mobility, but it requires a stopping rule to avoid infinite wandering.

Stochastic Hill Climbing

Instead of always selecting the single steepest improvement:

- ▶ identify neighbors that improve the objective;
- ▶ choose randomly among the available improving moves;
- ▶ repeat from the newly selected state.

Tradeoff

Randomness can explore different uphill trajectories and reduce deterministic trapping, but convergence may be slower.

Image placeholder

Landscape with several improving arrows and one randomly selected stochastic uphill move.

Key idea

Random choice among improving moves creates diversity without yet accepting worse states.

Best-First Hill Climbing

The source lecture describes a variant that examines successors incrementally rather than evaluating the full neighborhood.

- ▶ Generate candidate successors one at a time.
- ▶ Stop when the first improving neighbor is found.
- ▶ Move immediately to that state.
- ▶ Avoid scoring every successor when neighborhoods are large.

When it helps

This approach can reduce evaluation cost when there are many possible successors and improvements are common.

Image placeholder

Current state with many possible neighbors;
highlight the first sampled neighbor that
improves the objective.

Random-Restart Hill Climbing

Random restart runs hill climbing repeatedly from independently sampled initial states.

- ▶ Each run explores a different region of the landscape.
- ▶ A failed run does not force the next run to begin from the same local optimum.
- ▶ Continue until a satisfactory solution is found or a restart budget is exhausted.

Effective when

The state space contains many local optima and finding a sufficiently good solution matters more than following one deterministic trajectory.

Image placeholder

Multi-peak landscape with several random starting points and independent hill-climbing trajectories.

Key idea

Restarts trade additional time for robustness across difficult landscapes.

Part 4: Simulated Annealing — Sometimes Move the Wrong Way



Simulated Annealing

Definition

Simulated annealing combines local improvement with controlled random exploration by sometimes accepting a worse neighboring state.

At each step:

- ▶ select a random neighboring state;
- ▶ accept it if it improves the objective;
- ▶ if it is worse, accept it with a probability controlled by temperature;
- ▶ gradually reduce that temperature over time.

Physical analogy

Metallurgical annealing uses high heat for random motion, followed by gradual cooling toward a stable crystal structure.

Image placeholder

Annealing landscape showing an early trajectory that crosses out of a local optimum and later settles near a global optimum.

Simulated-Annealing Pseudocode

Algorithmic structure

```
current ← initial_state
for step = 1, 2, ...:
     $T \leftarrow \text{TEMPERATURE}(\text{step})$ 
    if  $T$  is sufficiently low: return current
    next ← random neighbor of current
    if next improves current:
        current ← next
    else if  $\text{ACCEPT\_WORSE}(\text{next}, \text{current}, T)$ :
        current ← next
```

Key idea

Early randomness can escape local traps; later low temperature makes the search increasingly conservative.

Temperature Controls Exploration

High temperature

- ▶ high randomness;
- ▶ worse moves are accepted more often;
- ▶ broad exploration of the state space.

Low temperature

- ▶ low randomness;
- ▶ worse moves are rarely accepted;
- ▶ search concentrates near promising states.

Cooling schedule

- ▶ fast cooling: quicker convergence, greater risk of getting stuck;
- ▶ slow cooling: more exploration and computation, greater opportunity to reach a better optimum.

Image placeholder

Temperature-versus-time curves comparing fast and slow cooling schedules.

Key idea

The cooling schedule determines how quickly simulated annealing shifts from exploration to exploitation.

Part 5: Population-Based Local Search

Local Beam Search

Definition

Local beam search maintains k candidate states in parallel instead of a single current state.

Each iteration:

- ▶ generate successors from all k current states;
- ▶ evaluate the combined successor pool;
- ▶ retain the best k states for the next iteration.

Beam width

$k = 1$ behaves like single-state hill climbing. Larger k increases breadth and exploration, but also computation and memory.

Image placeholder

Beam-search tree showing k retained states at each level while other successors are discarded.

Key idea

A population lets several regions of the state space compete at the same time.

Beam Search Can Lose Diversity

Premature convergence

If the best candidates become too similar, the beam may collapse into one region of the state space.

Mitigations from the source

- ▶ begin from diverse initial states;
- ▶ use a sufficiently wide beam;
- ▶ preserve enough variation for competing regions to survive.

Image placeholder

Two beam-search sketches: one collapsing to similar states and one maintaining diverse branches.

The “Hive Mind” Intuition

The source motivates population search using four ideas:

1. **Crowd**: generate many candidate solutions.
2. **Filter**: keep the candidates with the strongest fitness.
3. **Synthesis**: combine useful pieces from different candidates.
4. **Spark**: introduce a small random change to avoid a rut.

Image placeholder

Four-stage infographic: crowd of candidates → fitness filter → recombination → random mutation spark.

Key idea

Population methods replace one trajectory with repeated selection, recombination, and diversity.

Genetic Algorithms

Definition

A **genetic algorithm** maintains a population of candidate solutions and repeatedly applies fitness-based selection, crossover, and mutation.

Selection

Candidates with higher fitness are more likely to reproduce.

Crossover

Two parent candidates combine portions of their representations to produce offspring.

Mutation

Small random changes introduce new variation into the population.

Image placeholder

Genetic-algorithm cycle showing population → selection → crossover → mutation → next generation.

Key idea

Genetic algorithms perform global exploration by evolving a population rather than improving only one current state.

Genetic-Algorithm Pseudocode

Algorithmic structure

```
population ← RANDOM_POPULATION()
while stopping condition not met:
    evaluate FITNESS for every individual
    new_population ← [ ]
    while new_population is not full:
        parent1, parent2 ← SELECT(population)
        child ← CROSSOVER(parent1,parent2)
        child ← MUTATE(child)
        add child to new_population
    population ← new_population
return best individual
```

Key idea

Selection increases pressure toward strong solutions, while crossover and mutation generate new combinations for later generations.

Genetic Algorithms on Eight Queens

The source encodes an individual as eight row indices:

$$[r_0, r_1, \dots, r_7],$$

where index i is a column and r_i is the queen's row.

- ▶ **Initial population:** 100 random individuals.
- ▶ **Fitness:** number of non-attacking queen pairs.
- ▶ **Maximum fitness:** $\binom{8}{2} = 28$.
- ▶ **Selection:** tournament among 3 randomly selected individuals.
- ▶ **Crossover:** combine two parents at a random point.
- ▶ **Mutation:** with small probability (source example: 0.05), change one queen's row.

Image placeholder

8-queens genetic encoding with two parent vectors, crossover point, child vector, and a single-row mutation.

What Genetic Algorithms Trade

Strengths

- ▶ highly parallel population search;
- ▶ effective global exploration;
- ▶ useful when direct gradient information is unavailable or misleading;
- ▶ recombination can preserve useful traits from multiple candidates.

Limitations

- ▶ no general guarantee of convergence;
- ▶ performance depends on population size;
- ▶ crossover and mutation rates require tuning;
- ▶ populations can still lose diversity.

Key idea

Population diversity is valuable only if selection pressure does not eliminate it too quickly.

Part 6: Comparing Local-Search Strategies

Local Search at a Glance

Algorithm	Current states	Main exploration mechanism
Hill climbing	1	best local improvement
Stochastic hill climbing	1	random improving move
Random restart	1 per run	new random initial states
Simulated annealing	1	occasionally accept worse moves
Local beam search	k	retain best states from a population
Genetic algorithm	population	selection, crossover, and mutation

Key idea

These algorithms differ primarily in how much diversity they preserve and what kinds of non-greedy exploration they permit.

Exploration vs. Exploitation

Exploitation

Prefer states that already appear better.

- ▶ hill climbing;
- ▶ low-temperature annealing;
- ▶ strong fitness selection.

Exploration

Deliberately preserve alternatives or tolerate temporary regressions.

- ▶ stochastic choices;
- ▶ random restarts;
- ▶ high-temperature annealing;
- ▶ wider beams and mutation.

Key idea

Local-search design is largely the problem of balancing rapid improvement against the risk of premature convergence.

Is This AI?

Local search belongs to the classical AI toolkit.

- ▶ Like BFS, DFS, and A, it searches a space of possible solutions.
- ▶ Unlike path-finding algorithms, it emphasizes state quality rather than reconstructing a complete action sequence.
- ▶ Its mechanisms may feel systematic compared with modern generative AI, but the underlying search problems remain central to AI.
- ▶ The source highlights continued relevance in combinatorial optimization, robotics, and game AI.

Image placeholder

Classical-AI local-search iconography beside modern generative-AI imagery; include optimization, robotics, and game examples.

Key idea

AI includes both learned models and explicit search procedures for finding useful solutions in difficult state spaces.

Takeaways

- ▶ Local search focuses on the quality of complete states rather than storing explicit paths through a search tree.
- ▶ An optimization formulation requires a state space, an objective function, and a neighborhood function.
- ▶ Hill climbing is memory efficient but can become trapped by local optima, ridges, and plateaus.
- ▶ Sideways moves, stochastic choices, and random restarts increase robustness while retaining a single-state search model.
- ▶ Simulated annealing accepts controlled temporary regressions so that the search can escape local traps.
- ▶ Beam search and genetic algorithms maintain populations, using diversity to explore multiple regions at once.
- ▶ Next lecture: **Constraint Satisfaction Problems.**

Key idea

Local search succeeds by sacrificing systematic path exploration in exchange for memory efficiency, graded optimization, and controlled exploration.